

EXPERIMENT NO:1

```
def max_coins(piles):
    piles.sort()
    coins = 0
    n = len(piles)

    for i in range(n // 3, n, 2):
        coins += piles[i]

    return coins

piles = [2, 4, 1, 2, 7, 8]
```

Maximum coins: 9

=== Code Execution Successful ===

EXPERIMENT NO:2

```
def min_coins(coins, target):
    while current <= target:
        if i < len(coins) and coins[i] <= current:
            current += coins[i]
            i += 1
        else:
            current += current
            count += 1

    return count

coins = [1, 4, 10]
target = 19

print("Minimum coins needed:", min_coins(coins, target))
```

Minimum coins needed: 2

=== Code Execution Successful ===

EXPERIMENT NO:3

```
def minimum_time(jobs, k):
    jobs.sort(reverse=True)
    workers = [0] * k

    for job in jobs:
        index = workers.index(min(workers))
        workers[index] += job

    return max(workers)

jobs = [1, 2, 4, 7, 8]
k = 2

print("Minimum maximum time:", minimum_time(jobs, k))
```

Minimum maximum time: 11

=== Code Execution Successful ===

EXPERIMENT NO:4

```
def dijkstra(graph, source):
    n = len(graph)
    distance = [float('inf')] * n
    visited = [False] * n

    distance[source] = 0

    for _ in range(n):
        u = -1

        for i in range(n):
            if not visited[i] and (u == -1 or distance[i] < distance[u]):
                u = i

        visited[u] = True
```

Maximum profit: 120

=== Code Execution Successful ===

EXPERIMENT NO:5

```
def dijkstra(graph, source):
    n = len(graph)
    distance = [float('inf')] * n
    visited = [False] * n

    distance[source] = 0

    for _ in range(n):
        u = -1

        for i in range(n):
            if not visited[i] and (u == -1 or distance[i] < distance[u]):
                u = i

        visited[u] = True
```

[0, 7, 3, 9, 5]

=== Code Execution Successful ===

EXPERIMENT NO:6

```
import heapq

def dijkstra(n, edges, source, target):
    graph = [[] for _ in range(n)]

    for u, v, w in edges:
        graph[u].append((v, w))

    distance = [float('inf')] * n
    distance[source] = 0

    pq = [(0, source)]

    while pq:
        dist, u = heapq.heappop(pq)
```

Shortest path: 26

=== Code Execution Successful ===

EXPERIMENT NO:7

```
def huffman_codes(characters, frequencies):

    while len(tree) > 1:
        f1, left = heapq.heappop(tree)
        f2, right = heapq.heappop(tree)
        heapq.heappush(tree, (f1 + f2, (left, right)))

    def generate(node, code=""):
        if isinstance(node, str):
            codes[node] = code
        else:
            generate(node[0], code + "0")
            generate(node[1], code + "1")

    generate(tree[0][1])

{'a': '00', 'b': '01', 'c': '10', 'd': '11'}

=== Code Execution Successful ===
```

EXPERIMENT NO:8

```
def huffman_decode(characters, frequencies, encoded):
    for bit in encoded:

        if current in reverse:
            result += reverse[current]
            current = ""

    return result

characters = ['a', 'b', 'c', 'd']
frequencies = [5, 9, 12, 13]
encoded = "1101100111110"

print("Decoded message:", huffman_decode(characters, frequencies,
encoded))

Decoded message: aacda

=== Code Execution Successful ===
```

EXPERIMENT NO:9

```
def maximum_weight(weights, capacity):  
    weights.sort(reverse=True)  
  
    total = 0  
  
    for weight in weights:  
        if total + weight <= capacity:  
            total += weight  
  
    return total
```

```
weights = [10, 20, 30, 40, 50]  
capacity = 60
```

```
print("Maximum weight:", maximum_weight(weights, capacity))
```

Maximum weight: 60

=== Code Execution Successful ===

EXPERIMENT NO:10

```
def containers(weights, capacity):  
  
    for weight in weights:  
        if current + weight <= capacity:  
            current += weight  
        else:  
            count += 1  
            current = weight  
  
    return count
```

```
weights = [5, 10, 15, 20, 25, 30, 35]  
capacity = 50
```

```
print("Number of containers:", containers(weights, capacity))
```

Number of containers: 4

=== Code Execution Successful ===

EXPERIMENT NO:11

```
def kruskal(n, edges):  
    return mst, total
```

```
edges = [  
    (0, 1, 10),  
    (0, 2, 6),  
    (0, 3, 5),  
    (1, 3, 15),  
    (2, 3, 4)  
]
```

```
mst, total = kruskal(4, edges)
```

```
print("Edges in MST:", mst)  
print("Total weight:", total)
```

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight: 19

=== Code Execution Successful ===

EXPERIMENT NO:12

```
def find(parent, x):  
    if parent[x] != x:  
        parent[x] = find(parent, parent[x])  
    return parent[x]
```

```
def kruskal(n, edges):  
    parent = list(range(n))  
    mst = []  
  
    for u, v, w in sorted(edges, key=lambda x: x[2]):  
        a = find(parent, u)  
        b = find(parent, v)  
  
        if a != b:  
            parent[a] = b
```

Is MST unique? True

=== Code Execution Successful ===